IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Una red neuronal con solo NumPy

Neurona, activación, error y descenso del gradiente: abrir la caja antes de usar Keras

Sprint 10 · S10_01 · cuaderno: 40 min en clase (ejercicios 1, 2 y 3)

LA IDEA · SECCIÓN 1 · EJERCICIO 1

Una neurona: sumar con pesos y comprimir

Tres pasos: multiplica cada entrada por un **peso**, suma todo y añade un **sesgo**, y pasa el resultado por una **función de activación**, un «interruptor suave».

Entrada	valor	peso	valor × peso
Antelación (cientos de días)	1,5	0,8	1,2
Tarifa flexible (1 = sí)	1,0	1,2	1,2
Suma + sesgo (−1,0)			z = 1,4

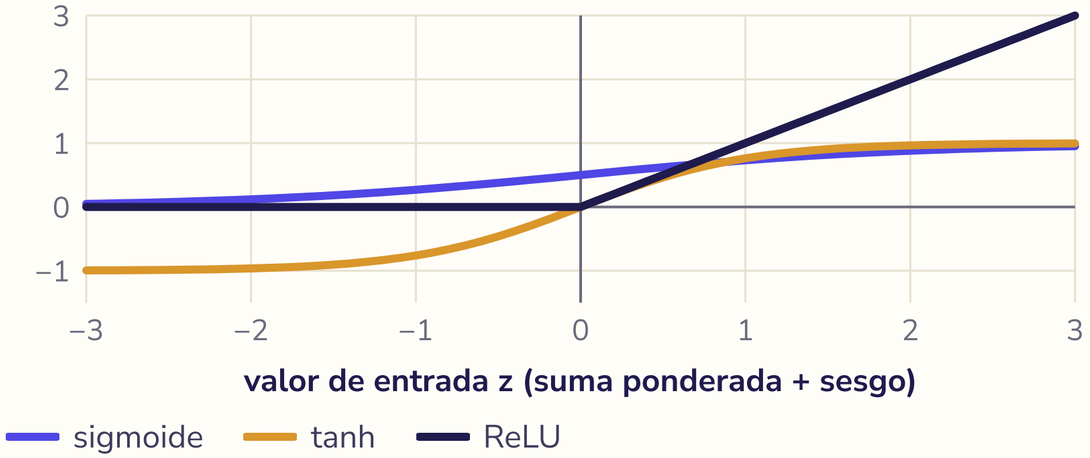
```
def neurona(x, pesos, sesgo):
    return sigmoide(np.dot(x, pesos) + sesgo)
```

0,802

sigmoide(1,4): la neurona da un **80 %** de probabilidad de cancelar.

LA IDEA · SECCIÓN 2 · EJERCICIO 2

Sin curvas, apilar capas no sirve de nada



z	sigmoide	tanh	ReLU
−2	0,12	−0,96	0
0	0,50	0,00	0
2	0,88	0,96	2

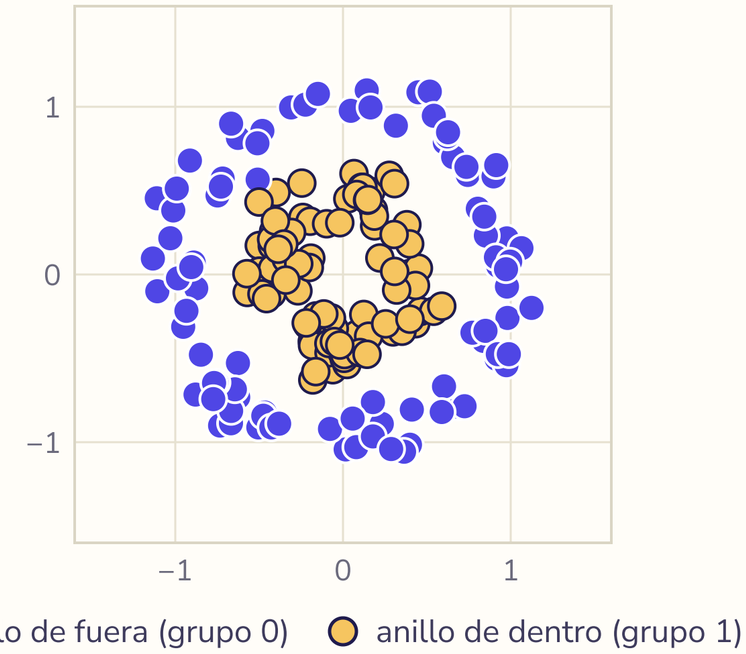
```
relu(z) = np.maximum(0, z)
relu_derivada(z) = (z > 0).astype(float)
```

Una suma de sumas ponderadas sigue siendo una suma ponderada: sin activación, la red es un modelo **lineal**.

EL PROBLEMA · SECCIÓN 3

Dos anillos que ninguna recta separa

180 puntos de validación (make_circles)



Regresión logística (una recta)

Exactitud en validación: **49,4 %**. Es como tirar una moneda: cualquier recta deja la mitad de cada anillo a cada lado.

La red que vas a construir

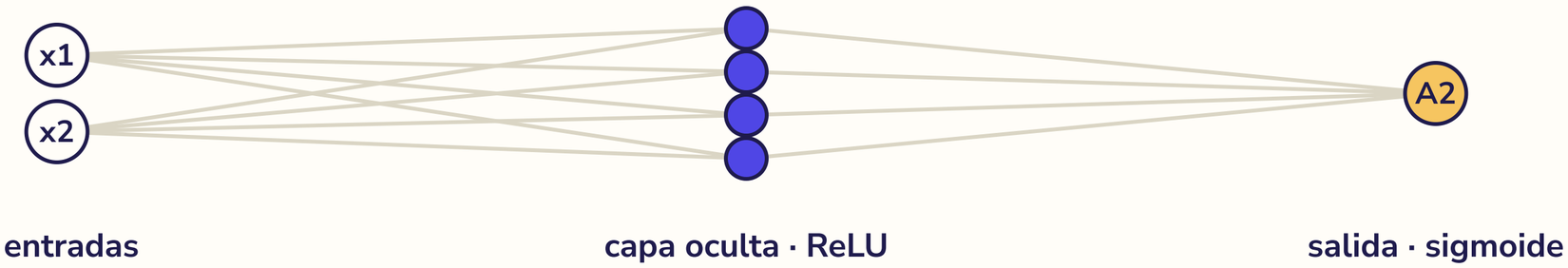
2 entradas → capa oculta (ReLU) → 1 salida (sigmoide).

```
X, y = make_circles(n_samples=600,
                    noise=0.10, factor=0.45,
                    random_state=42)
```

APRENDER · SECCIÓN 4 · HACIA DELANTE

De las entradas a la predicción

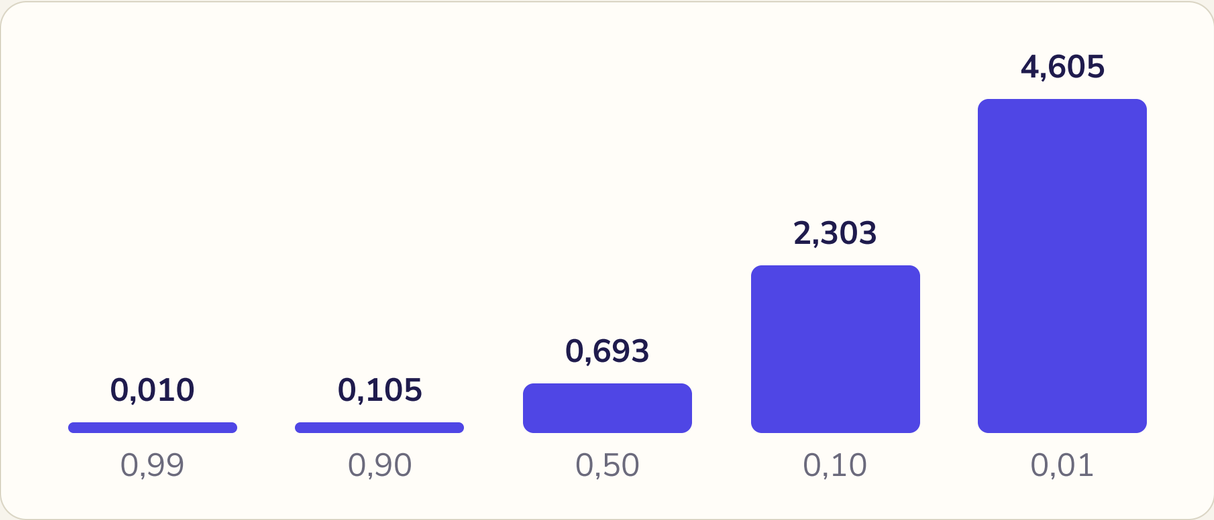
Una capa son muchas neuronas a la vez: se calcula con **una multiplicación de matrices**.




```
Z1 = X @ W1 + b1     # suma ponderada de la capa oculta
A1 = relu(Z1)        # activación
Z2 = A1 @ W2 + b2    # suma ponderada de la salida
A2 = sigmoide(Z2)    # probabilidad de ser del grupo 1
```

La pérdida castiga fallar con seguridad

Entropía cruzada: mira la probabilidad que la red dio a la **respuesta correcta**. Menor es mejor.



Probabilidad a lo correcto → pérdida

0,5 (no saber nada) es ln 2 = 0,693.

Si acierta con seguridad (0,99), la pérdida es **0,010**; si falla con seguridad (0,01), **4,605**.

```
entropia_cruzada(np.array([1, 0]), np.array([0.99, 0.01]))   # 0.01
entropia_cruzada(np.array([1, 0]), np.array([0.01, 0.99]))   # 4.605
```

El gradiente: ¿sube o baja el error si muevo un peso?

Mini-red de juguete: 1 entrada, 1 neurona oculta (ReLU) y 1 salida (sigmoide). Una reserva con **x** = 2 que **sí cancela** (**y** = 1) y pesos w1 = 0,5, b1 = 0,0, w2 = 1,0, b2 = −0,5.

Hacia delante →	cálculo	valor
z1	2 · 0,5 + 0	1,00
a1 = ReLU		1,00
z2	1,00 · 1,0 − 0,5	0,50
a2 = sigmoide		0,622
error	−ln(0,622)	0,474

← Hacia atrás	cálculo	valor
dz2	a2 − y	−0,378
dw2 y **db2**	a1 · dz2 ; dz2	−0,378
dz1	dz2 · w2 · 1 (z1 > 0)	−0,378
dw1	x · dz1	−0,755
db1	dz1	−0,378

El error de la salida (a2 − y) viaja hacia atrás: todos los gradientes son negativos, así que subir los pesos baja el error.

APRENDER · EJERCICIO 3

Un pasito en contra del gradiente

Parámetro	valor	gradiente	nuevo = valor − 0,1 · gradiente
w1	0,500	−0,755	0,576
b1	0,000	−0,378	0,038
w2	1,000	−0,378	1,038
b2	−0,500	−0,378	−0,462

```
def paso_gradiente(parametros, gradientes, tasa):
    return {k: parametros[k] - tasa * gradientes[k]
            for k in parametros}
```

0,474 → 0,380

El error de esta reserva baja tras **un solo** paso.

adelante → error → atrás → actualizar. Repetirlo con todos los datos es una época.

PARA CASA · SECCIÓN 5

La red completa y su prueba de gradientes

adelante

Calcula **Z1**, **A1**, **Z2** y **A2**.

atras

Lleva el error de la salida, **A2 – y**, hacia la entrada.

entrenar

Adelante → atrás → **paso_gradiente**, época tras época.

```
red = RedNeuronal(2, 8).entrenar(X_ent, y_ent, epocas=1500, tasa=0.3, X_val=X_val, y_val=y_val)
```

0,002669

gradiente de **W1[0, 1]** por **retropropagación**

0,002669

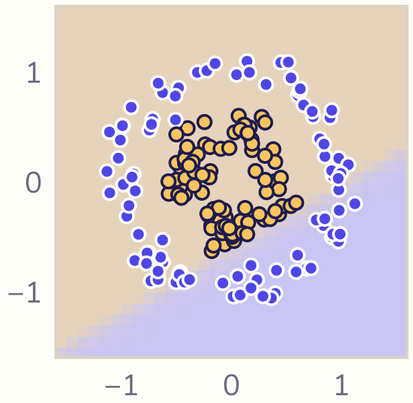
gradiente **numérico**: mover el peso ±0,00001

Coinciden: la propagación hacia atrás está bien programada.

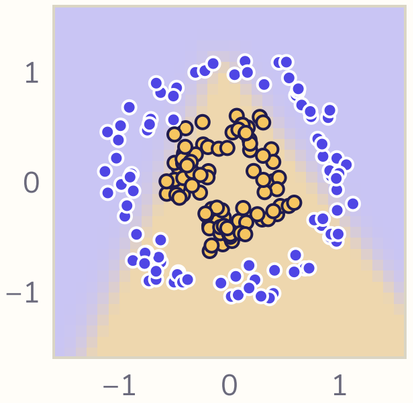
PARA CASA · SECCIÓN 6 · EJERCICIO 4

Más neuronas, una frontera más fina

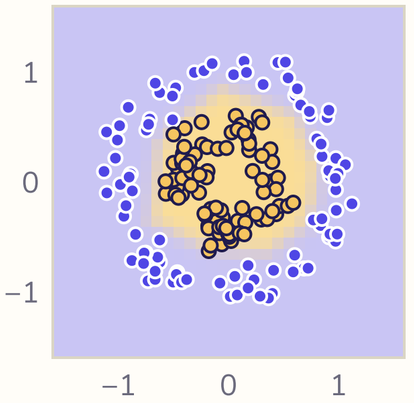
1 neurona oculta · 65,6 %



2 neuronas ocultas · 77,2 %



8 neuronas ocultas · 100 %



Fondo dorado: la red predice «grupo 1»; fondo lavanda: «grupo 0». Los puntos son los de validación.

```
exactitud_val = (mejor.predecir(X_val) == y_val).mean()            # 8 neuronas: 1.0
mejora_sobre_lineal = exactitud_val - lineal.score(X_val, y_val)   # 1.000 - 0.494 = 0.506
```

PARA CASA · SECCIÓN 7 · MINI-RETO

La tasa de aprendizaje manda

Tasa (4 neuronas, 1 500 épocas)	Pérdida de validación: época 0 → 1 499	Exactitud de validación
0,3	0,716 → 0,025	100 %
0,01	0,721 → 0,621	60,6 %
50	1,122 → 5,944 (oscila)	50,0 %

Muy pequeña: lenta

Con 0,01 la red apenas ha empezado a bajar la pérdida tras 1 500 épocas.

Enorme: inestable

Con 50 los pesos saltan de un lado a otro: la pérdida **se dispara** y no aprende.

```
red_lenta = RedNeuronal(2, 4).entrenar(X_ent, y_ent, epocas=1500,
                                       tasa=0.01, X_val=X_val, y_val=y_val)   # 0.606
```

HOY HAS APRENDIDO

Neuronas en cadena que aprenden

Neurona y activación

Suma ponderada + sesgo + activación. Sin curvas, la red sigue siendo **lineal**

Aprender

Adelante → error → atrás → actualizar. El **gradiente** dice hacia dónde mover cada peso

Capacidad y tasa

Pocas neuronas no cierran la frontera; pequeña = lento, enorme = inestable

Siguiente: S10_02 · la misma idea con Keras